同济大学 TONGJI UNIVERSITY

第二次培训：工作空间、编译与三条技术路线

ROS 2 工作空间实操 · 编译排错 · 视觉巡线 / Nav2 加 TEB / 二维 LIO

智能车培训

2026-10-13

同济大学电子与信息工程学院

目录

先分清四个目录 1
一、工作空间与包的创建 1
包是什么，必须放哪里 2
创建一个包 3
两种包的最小结构 4
依赖写在 package.xml 5
实操：把三个 demo 包放进 src/ 6
编译只有一个正确位置 7
二、编译：位置、顺序与三类典型报错 7
三条命令和顺序 8
两个常用参数 9
故障一：系统时间错 10
故障一：认这几个关键词 11
故障一：怎么修 12
故障二：编译错位置 13
故障二：先把产物删干净 14
故障二：备用手段 15
故障三：忘了 source install . . 16
排错总表 17
三条路线共用一套硬件 18
三、三条技术路线概览 18
三条路线对比 19
demo1 视觉巡线：一条闭环 . 20
demo2 Nav2 加 TEB：搭起一条链 21
demo3 二维 LIO：自己算位姿 22
三条路线的公共约束 23
技术栈对照 24
四、技术栈与难度评估 24
难度评估 25

各自难在哪 26
这份评估的依据 27
开工前的四条自查 28

一、工作空间与包的创建

一个 ROS 2 工作空间，最外层只有四个目录。它们的分工不一样，删错了代价也不一样。

目录	装什么	能不能删
src/	包的源代码，你唯一需要手工编辑的地方	不能，删了就没了
build/	编译中间产物，CMake 缓存和目标文件	能，会自动重建
install/	安装结果：可执行文件、launch、config，还有每个包的 package.xml 副本	能，会自动重建
log/	每次编译的日志，排错先看这里	能，建议按需清

编译产物的价值只有一个：能被重新生成。所以遇到说不清的问题，删掉重编通常比逐个排查更快。

包是什么，必须放哪里

`package.xml` 是包的身份证。colcon 从当前目录往下递归找 `package.xml`，找到哪个目录，哪个目录就是一个包。

由此得出两条硬规则。

- 包必须放在 `src/` 下面，colcon 从工作空间根开始扫描，`src/` 之外的东西不会被发现。
- 包的位置只影响“能不能被找到”，不影响目录深度。惯例是 `src/<包名>/`，多一层嵌套也能被找到。

创建一个包

```
cd /userdata/dev_ws
source /opt/tros/humble/setup.bash

# C++ 包
ros2 pkg create --build-type ament_cmake my_cpp_pkg \
  --dependencies rclcpp geometry_msgs

# Python 包
ros2 pkg create --build-type ament_python my_py_pkg \
  --dependencies rclpy
```

命名规则：全小写，单词之间用下划线，不要用大写和连字符。`demo1_visual_control` 合规，`Demo1-VisualControl` 不合规。

两种包的最小结构

C++ 包（ament_cmake）

```
my_cpp_pkg/
  package.xml
  CMakeLists.txt
  src/xxx.cpp
  include/my_cpp_pkg/xxx.hpp
  launch/xxx.launch.py
  config/xxx.yaml
```

编译脚本里必须写清楚三件事：`find_package`、`add_executable`、`install`。

Python 包（ament_python）

```
my_py_pkg/
  package.xml
  setup.py
  setup.cfg
  resource/my_py_pkg
  my_py_pkg/xxx.py
  launch/xxx.launch.py
  config/xxx.yaml
```

`setup.py` 里的 `entry_points` 决定可执行名字，`data_files` 决定 launch 和 config 装到哪。

依赖写在 package.xml

用到别人的消息或库，就要在 `package.xml` 里声明。漏了会在两个阶段报错，位置不同，原因相同。

阶段	典型报错	缺的是什么
编译	`fatal error: geometry_msgs/msg/twist.hpp: No such file or directory`	`<depend>` 没写，或 CMake 里没 `find_package`
链接	`undefined reference to ...`	库找到了但没链接，检查 `ament_target_dependencies`
运行	`Package 'x' not found` 或 import 失败	`<exec_depend>` 没写，或忘了 source install

C++ 包还要在 `CMakeLists.txt` 里做两件对应的事：`find_package(...)` 和 `ament_target_dependencies(可执行名 依赖名)`。

实操：把三个 demo 包放进 src/

```
cd /userdata/dev_ws
cp -a <training>/demos/demo1_visual_ws/src/demo1_visual_control src/
cp -a <training>/demos/demo2_nav2_teb_ws/src/demo2_nav2_teb src/
cp -a <training>/demos/demo3_lio_ws/src/demo3_lio src/
```

只复制包目录。外层 `*_ws`、`build/`、`install/`、`log/` 都不要复制，复制进去就会带来别人机器上的编译产物。

二、编译：位置、顺序与三类典型报错

编译地点是“包含 src/ 的那一层”，也就是工作空间根。不是 src/ 里面，也不是某个包目录里面。

```
pwd                 # /userdata/dev_ws
ls src              # 应该能看到 demo1_visual_control 等包名
```

这两条命令能同时成立，位置就是对的。判断依据只有一个：colcon 在哪个目录被调用，build/、install/、log/ 就生成在哪个目录。

三条命令和顺序

```
cd /userdata/dev_ws                          # 1. 进工作空间根
source /opt/tros/humble/setup.bash           # 2. 系统环境
colcon build --symlink-install \
  --packages-select demo1_visual_control     # 3. 编译
source install/setup.bash                    # 4. 让系统认识新编译的包
```

第 2 步和第 4 步是两件事。前者是系统和厂商包，后者是你刚编译出来的包。ROS 2 通过环境变量 `AMENT_PREFIX_PATH` 找包，不执行第 4 步，`ros2 pkg list` 里就没有你的包。

每开一个新终端，第 2 步和第 4 步都要重做一遍。bashrc 里已经有的只是第 2 步。

两个常用参数

参数	作用	常见误解
`--packages-select <包名>`	只编译指定包，跳过无关包，省时间	它不会自动编译依赖，依赖缺了仍然报错
`--symlink-install`	Python 代码、launch、config 用软链接安装	C++ 代码改完仍然必须重新编译

只改 Python 或 YAML 时，`colcon build` 之后不用重新复制文件，改完直接重启节点就生效。改 C++ 一定要重新编译，否则跑的还是旧程序。

故障一：系统时间错

这台车每次开机的默认时间是 2000 年。我们已经设置了联网自动同步，但开机到同步完成之间有一个窗口，这时候编译就会出错。

先看时间，一秒确诊：

```
date
```

输出是 2000 年，后面所有编译问题都不用查了，先同步时间。

故障一：认这几个关键词

时间不对时，报错不在代码里，而在工具对文件时间戳的信任上。见关键词就对号入座。

```
make: warning: Clock skew detected. Your build may be incomplete.
make: Warning: File 'xxx.o' has modification time 8.2e+08 s in the future
E: Release file for http://... is not valid yet (invalid for another 9d 3h)
Certificate verification failed: The certificate is NOT yet valid
certificate is not yet valid
```

最后两条来自 `apt`、`rosdep` 或 `pip` 联网。证书有生效时间，系统时钟比证书的生效时间还早，就会报“证书尚未生效”。

故障一：怎么修

```
sudo timedatectl set-ntp true
sudo timedatectl status       # 看 System clock synchronized: yes
date                          # 确认年份已经是今年
```

没有网络时，先手工设一个接近当前的时间，再继续：

```
sudo date -s "2026-10-13 19:30:00"
```

然后重新编译。已经用错时间编译出来的产物，直接删掉重来更干净。

故障二：编译错位置

如果有人在 `src/` 里面或者某个包目录里执行了 `colcon build`，产物就落在那里，变成 `src/build`、`src/install`、`src/log`。

麻烦在于 `install/` 里面有 `share/<包名>/package.xml`，colcon 下次扫描时会把这份安装结果也当成一个包。于是同一个包名出现两次。

```
Duplicate package names not supported
CMake Error: The current CMakeCache.txt directory /a/build/x is different
  than the directory /b/build/x where CMakeCache.txt was created
```

第二种报错来自“同一份源码被两个不同路径编译过”，通常是复制工作空间或复制 `build/` 造成的。

故障二：先把产物删干净

删之前确认自己在工作空间根，要删的对象只有 build、install、log。

```
cd /userdata/dev_ws
pwd
find src -maxdepth 3 -type d \
  \( -name build -o -name install -o -name log \)
rm -rf src/build src/install src/log
rm -rf build install log
colcon build --symlink-install --packages-select <包名>
source install/setup.bash
```

src/ 里的代码一行都不会被删。删完必须重新 source install/setup.bash，否则终端里的环境还指向已经被删掉的旧路径。

故障二：备用手段

万一某个目录暂时不能删（比如里面有别人还没提交的改动），可以在那个目录里放一个空的 `COLCON_IGNORE` 文件。colcon 扫描时看到这个文件就跳过该目录。

```
touch src/build/COLCON_IGNORE
```

这只是临时绕开，事后仍然应该把产物删掉。留着的坏处是以后每次编译都会被跳过，容易忘记。

故障三：忘了 source install

```
$ ros2 launch demo1_visual_control demo1.launch.py
Package 'demo1_visual_control' not found
```

包已经编译好了，只是当前终端不知道它在哪。处理：

```
source /userdata/dev_ws/install/setup.bash
ros2 pkg list | grep demo          # 能看到三个 demo 包
```

辅助检查：`echo $AMENT_PREFIX_PATH | tr ':' '\n'` 会列出当前终端能看到的全部包路径。

排错总表

报错里的关键词	说明	处理
`Clock skew detected`	系统时间不对	先 `date`，再 `timedatectl set-ntp true`
`in the future`	文件时间戳在未来	同上，然后删产物重编
`is not valid yet` / `NOT yet valid`	证书尚未生效，时钟太早	同上
`Duplicate package names not supported`	同一包名被扫描到两次	删掉 `src/` 里的 build、install、log
`CMakeCache.txt ... is different`	build 目录来自另一个路径	删 `build/` 重编
`Package 'x' not found`	当前终端没 source install	`source install/setup.bash`
`No such file or directory` 指头文件	依赖没声明	补 `<depend>` 和 `find_package`
`undefined reference`	没链接库	补 `ament_target_dependencies`

三、三条技术路线概览

```
底盘轮速 + IMU ──> EKF ──> odom 到 base_footprint (TF)
                          └─> /odom、/odom_combined

N10 雷达 ──> /scan

USB 相机 ──> YOLO ──> /win_detection
```

```
三条控制器 ──> /cmd_vel ──> 底盘驱动 ──> STM32 串口
```

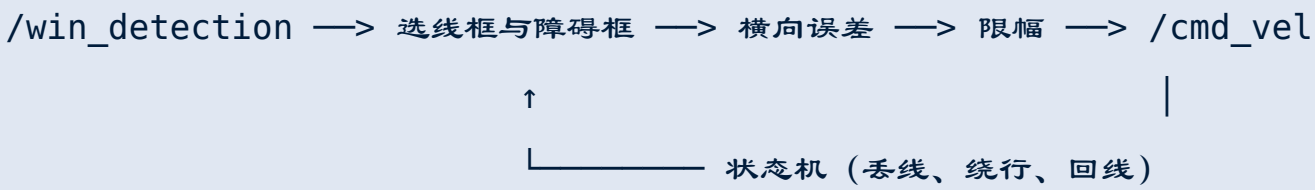
三条路线的区别只有三点：位置从哪里来，障碍从哪里来，谁发 /cmd_vel。

三条路线对比

路线	位置来源	障碍来源	谁发 /cmd_vel	车端对应实现
demo1 视觉巡线	图像里检测框的横坐标	YOLO 检测框	视觉控制器	`origincar_control`
demo2 Nav2 加 TEB	录制路径、TF、定位	雷达局部代价地图	TEB 控制器插件	`my_robot_nav` 加 `dzsavepath`
demo3 二维 LIO	自己算的二维位姿	雷达点和车体包络	纯追踪控制器	`lio_hybird`

三条路线的教学包分别是 `demo1_visual_control`、`demo2_nav2_teb`、`demo3_lio`。它们都是从车端实现里裁出来的骨架，接口一致，核心实现挖空。

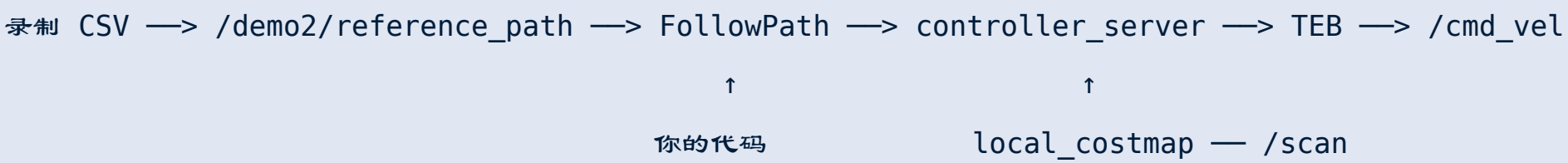
demo1 视觉巡线：一条闭环



要写五处：`best_detection()` 解析多框、`make_command()` 算误差与限幅、障碍物多帧确认与绕行方向锁存、`lost_line_command()` 丢线保持与搜索、`reset_pid()` 的调用点。

难度不在算法。图像横坐标正方向与角速度正方向是否一致、时间间隔不恒定、参考列是 920 而不是图像中心的 960，这三件事都要在真实数据上确认。

demo2 Nav2 加 TEB：搭起一条链



要补三样：一张真实地图、nav2_full_params.yaml 里的全部参数（含 controller_server.TEB 的 80 个参数）、FollowPath 客户端的发送与恢复逻辑。

启动分三级台阶：只发路径看话题，再起 Nav2 不发目标，最后自己打开 student_ready 安全门。

demo3 二维 LIO：自己算位姿



要写六件事：按束时间戳去畸变、过滤越界点与孤立点、雷达外参变换、用里程计与 IMU 预测、ICP 或 NDT 配准加观测拒绝门限、把占位输出换成真位姿。

验收最硬：静止 60 s 位姿不发散，直线 5 m 往返有误差统计，原地转动时墙体不明显旋转，插入坏点后不崩溃。

三条路线的公共约束

同一时间只能启动一条路线。三个控制器都会写 /cmd_vel，一起启动就是三个控制器抢一台车。

```
ros2 topic info /cmd_vel -v      # 正常情况下只有一个发布者
```

约束	说清楚的问题	踩错的现象
控制权唯一	只有一条路线的节点在发 /cmd_vel	车按另一套逻辑走
TF 归属	`odom 到 base_footprint` 由 EKF 发布，别人不能重复发	TF 出现两个父节点，Nav2 查询失败
时间源一致	回放带 `/clock` 的录包时全部节点加 `use_sim_time`	TF 报 extrapolation，路径与雷达错位
frame 名字	IMU 的 frame 是 `gyro_link`	tf2 查询报 frame 不存在

四、技术栈与难度评估

路线	语言	ROS 2 知识点	领域知识	外部依赖
demo1	C++	话题、QoS、参数、定时器	像素到控制量的映射、比例控制与 PID、状态机	相机与 YOLO 推理节点
demo2	Python 加 YAML	action、lifecycle、TF、参数层级、插件加载	代价地图、路径跟踪、优化器调参	Nav2 全套、TEB 插件、真实地图
demo3	C++	话题、QoS、TF、时间戳与时间同步	平面状态估计、去畸变、ICP 与 NDT、观测拒绝	雷达、IMU、轮速里程计、Eigen

共同的基础是同一套：Linux 命令行、ROS 2 的节点与话题、TF 树的父子关系、rosbag 的录制与回放。这部分在三条路线上完全复用。

难度评估

维度	demo1 视觉巡线	demo2 Nav2 加 TEB	demo3 二维 LIO
自己要写的代码量	约 200 至 350 行 C++	约 100 至 250 行 Python，其余是配置	约 500 至 1000 行 C++
算法深度	低，比例控制加状态机	低，读框架不写算法	高，去畸变、ICP、NDT、协方差
主要难点	符号约定与实车调参	插件版本、TF 归属、参数类型	时间同步与观测拒绝
依赖可控性	高，有录包可以离线练	低，插件、地图、参数都要对齐	中，需要真实传感器数据
验收客观性	中，靠现象观察	中，靠现象观察	高，有硬指标
入门门槛	最低	中等	最高

各自难在哪

demo1 的算法最浅，物理侧要求最高。符号错了车会越跑越偏，参数不对会在弯道上来回震荡，两个问题都只能在真实数据上试出来。

demo2 自己写的代码最少，集成量最大。地图、参数、插件版本、TF 的每条边、代价地图的输出，任何一处不对都起不来，而报错信息通常不指向真正的原因。

demo3 算法最重，也最接近科研。时间对齐、运动补偿、外参、配准与拒绝，每一环都能单独让结果发散。它的好处是验收有硬指标，做没做对可以离线复现。

这份评估的依据

评估来自三样可核对的东西：车端实现的体量、教学骨架留下的接口、以及三条路线对环境的依赖程度。

	车端实现	教学骨架	差额在哪
demo1	`origincar_control` 966 行	约 150 行外壳加占位	四类目标解析、绕行方向推断、回线特判、二维码接管
demo2	参数文件 412 行，加 `dzfollowpath`	路径发布约 120 行	地图、参数取值、行为树、FollowPath 完整逻辑
demo3	LIO 节点 64 KB，纯追踪节点 208 KB	两个节点合计约 10 KB	去畸变、ICP、NDT、观测拒绝、协方差、keepout

这不是性能实测，而是根据代码体量和接口宽度做的估计。真正的难度会在你动手的第二天体现出来。

开工前的四条自查

```
date                                          # 1. 时间对不对，先看这一条
source /opt/tros/humble/setup.bash            # 2. 系统环境
source /userdata/dev_ws/install/setup.bash    # 3. 工作空间环境，路径按实际改
ros2 pkg list | grep demo                     # 4. 三个包在不在
```

再加一条上车前的检查：

```
ros2 topic info /cmd_vel -v                   # 发布者只能有一个
```

四条都过了再动手，前三条能挡掉这节课里绝大多数“编译不过”。



同濟大學
TONGJI UNIVERSITY

谢谢

下次动手前，先 date，再编译。